PROGRAM EXPLANATION

プログラム説明資料

Action3D

自作フレームワークで動く 3D アクション

制作期間　2026年6月 –(開発中)

制作形態　個人開発

開発環境　C++ / DirectX 11 / HLSL

菊池 凌斗

Ryoto Kikuchi / HAL東京 ゲーム学科



開発情報 — 期間・体制・環境・制作意図

開発期間	2026年6月～（開発中）。約4ヶ月
所要時間（概算）	約65時間（週に約4時間 × 約16週）
開発メンバー	1名（個人開発）
自身の担当範囲	企画・設計・調整・動作確認のすべて。コードの記述は主に AI（Claude Code）に依頼した（次ページに詳細）
制作意図	「最高に気持ちいいコンボアクション」を作りたかった。そのため、キャンセル表・先行入力・ジャスト回避・ヒットストップなど、手触りに関わる部分を作り込んだ
動作環境	Windows 11 / Intel Core i7 / GeForce RTX 3050 Laptop GPU（VRAM 6GB、開発機で動作確認）。DirectX 11 対応の GPU が必要。実行ファイルは bin/Release、ビルドは Visual Studio 2022（x64）
使用技術	C++ / DirectX 11 / HLSL。XAudio2・X3DAudio・Media Foundation。DirectXTex・cgltf・Dear ImGui・SDL3（ゲームパッド入力、静的リンク）・stb_truetype。**既存のゲームエンジンは使用せず、フレームワークから自作**

操作方法と、AI・外部素材の利用

操作方法

移動	ＷＡＳＤ　/　パッド: 左スティック
視点	マウス　/　パッド: 右スティック
弱攻撃	マウス左クリック　/　パッド: 左ボタン
強攻撃	マウス右クリック　/　パッド: 上ボタン
ジャンプ	Space　/　パッド: 下ボタン
ダッシュ・回避	Shift（押しっぱなしでダッシュ）、Ctrl（回避）　/　パッド: R2（押しっぱなしでダッシュ、押して回避）
ロックオン	マウスのホイールクリック　/　パッド: L2
雷（範囲攻撃）	E　/　パッド: R1
触手（敵を集める）	Q　/　パッド: L1
触手（敵へ飛びつく）	R　/　パッド: 右ボタン
ポーズ	Esc　/　パッド: Start

AI の利用について

コード: 設計・仕様の決定・調整・動作確認は私が行い、**コードの記述は主に AI（Claude Code）**に依頼した。仕様を指示し、動作を確認したうえで採用している。

アセット（3D モデル・テクスチャ）: 画像生成 AI で元絵を作り、Tripo AI で 3D 化し、Mixamo でリグ・アニメーションを付け、Blender で調整した。

音声（BGM・SE）: CC0（著作権放棄）の素材、または AI が生成した音源を使用している。

フォント: BIZ UDPGothic（SIL Open Font License）。

ライブラリ: 左のページの「使用技術」のとおり。ライセンス文は `source/licenses` に同梱している。

作品概要と開発環境

作品概要

DirectX 11 の API を直接使い、描画・物理・
アセット管理・オーディオまでを自作した
フレームワークと、その上で動く3Dアクションゲーム。

ゲーム側は、剣と「虚装」(体から伸びる影の武器)で敵の群れとボスと戦う。

開発環境

言語 / API	C++ / DirectX 11(HLSL)
ライブラリ	DirectXTex / cgltf / Dear ImGui / SDL3 / stb_truetype
音声	XAudio2 / X3DAudio / Media Foundation
期間・形態	2026/6 –(開発中)・個人開発
私の役割	設計・調整・動作確認(コードの記述は主に AI を利用)。素材は外部ツールで用意



素材は 画像生成 → Tripo AI → Mixamo → Blender の流れで用意し、実装に時間を使う体制にした

コードの構成 — フレームワーク層とゲーム層

フレームワーク層 src/engine/ 75ファイル・13,184行

フォルダ	役割	数	行数
engine/asset	アセット管理・VFS・ベイク(pak / DDS / PCM / MDL)	13	1,539
engine/audio	XAudio2 と X3DAudio による再生・3D 音響	3	1,546
engine/component	GameObject と Component の土台	3	128
engine/core	シーンとオブジェクトの管理・時間	8	773
engine/debug	デバッグ描画・計測(Profiler)	3	214
engine/feedback	ヒットストップ・カメラ演出・画面効果	10	620
engine/graphics	DirectX 11 描画・glTF・カメラ・文字描画	16	4,606
engine/input	キーボード・マウス・ゲームパッド	10	2,294
engine/math	数学(ベクトル)	1	114
engine/physics	衝突判定(コライダー・レイヤー)	8	1,350

ゲーム層 src/game/ 84ファイル・13,782行

フォルダ	役割	数	行数
game/actors	プレイヤー・敵・弾・斬撃	14	4,098
game/app	ゲーム進行・チュートリアルほか	9	1,171
game/effects	爆発・虚装(触手)・パーティクルなど	23	4,222
game/ui	HP バー・スコア・ウェーブ表示・ポーズ	16	1,524
game/weapon	武器(剣・虚装)	7	1,879
game/world	地形・街・木・草・空	15	888

● 単独 個人開発(チームなし)。設計と調整は私、コードの記述は主に AI(Claude Code)。入力の `input_manager` と `game_controller` は私が実装した。

合計 159 ファイル・26,966 行(.cpp / .h、コメント・空行を含む)。ほかに HLSL シェーダー29本。外部ライブラリは `external/` に分けて置いている。
「フレームワーク層」は他のゲームでも使える部品、「ゲーム層」はこの作品専用のコード、という切り分け。

アセットの追加を「1行」で済ませる — X-Macro と AssetManager

課題

テクスチャやモデルを1つ足すたびに、`enum`・パスの配列・キャッシュの3か所を同じ順番で直す必要がある。順番がずれると、別のアセットが読み込まれる。

やったこと

定義を `asset_def.h` の1か所に集約した❶。`TEXTURE(ID, パス)` を1行書くだけで、マクロを差し替えて展開し、`enum` ❷とパスの配列❸を同じリストから作る(X-Macro)。

`AssetManager` は ID から取得するだけの窓口。一度作ったテクスチャはキャッシュし、起動時に `PreloadAll` でまとめて読む。

ポイント

- Release ではベイク済みの `.dds` を読む(デコードもミップ生成も無し)❹。Debug は生の png / hdr を読み、その場でミップを作る。
- 1行の追加で済むので、アセットの追加時にバグが入り込まない。

asset_def.h

```
15  #define TEXTURE_LIST \                                          ❶
16      TEXTURE(FIELD,      "assets/textures/environment/field.png") \
17      TEXTURE(FIELD_NORMAL,   "assets/textures/environment/field_normal.png") \
    ⋮
32  #define TEXTURE(id, path) id,
33  enum class TextureID {                                          ❷
34      TEXTURE_LIST
35      MAX
36  };
```

asset_def.h L15–L17 / L32–L36

asset_manager.cpp

```
21  #define TEXTURE(id, path) path,
22  static const char* g_texturePaths[static_cast<int>(TextureID::MAX)] = {   ❸
23      TEXTURE_LIST
24  };
25  #undef TEXTURE
    ⋮
95      if (AssetFileSystem::IsPackMode()) {
96          //--- Release: ベイク済み .dds を読む（デコード・ミップ生成なし）---   ❹
97          std::string path = origPath;
98          size_t dot = path.find_last_of('.');
99          if (dot != std::string::npos) path = path.substr(0, dot) + ".dds";
100
    ⋮
118     } else {
119         //--- Debug: 生 png/hdr をランタイムでデコード＋ミップ生成 ---
120         std::vector<uint8_t> fileData;
121         if (!AssetFileSystem::ReadFile(origPath, fileData)) return nullptr;
```

asset_manager.cpp L21–L25 / L95–L100 / L118–L121

起動を速くする — 自作のアセットベイクと VFS

課題

起動のたびに、MP3・PNG のデコード、ミップ生成、GLB のパースが走り、起動が遅い。

やったこと

ゲーム起動時、`data.pak` が無い・古いときだけ**ベイク**を実行し、1つの pak にまとめる。

- MP3 は**PCM にデコード**してから格納する❸。
- PNG / HDR は**ミップ付きの DDS** にする❹(非 HDR・非法線は BC7 圧縮)。
- GLB は**ベイク済みの .mdl** にする。頂点・ボーン・アニメーションのキーを、**GPU へコピーするだけの形へ事前変換**しておく(ワインディング反転などもここで焼き込む)。テクスチャは BC7。

ポイント

- 作り直すかは、pak の**フォーマット版数**❶と、ソースの**更新時刻**❷で判定する。
- pak の先頭は、作っている間は `"PAK?"` にしておき、書き終えてから `"PACK"` に戻す❺。途中で落ちても壊れた pak を読ませない。

asset_baker.cpp

```
63          if (n != 1 || memcmp(h.magic, "PACK", 4) != 0 || h.version != PackFormat::VERSION) {   ❶
64              return true;
65          }
    ⋮
72  auto pakTime = fs::last_write_time(pakPath);
73
74  // srcDir 内を再帰スキャンして、pak より新しいファイルがあるかチェック
75  for (auto& entry : fs::recursive_directory_iterator(srcDir)) {
76      if (!entry.is_regular_file()) continue;
77      if (entry.last_write_time() > pakTime) return true;                                          ❷
78  }
    ⋮
581 memcpy(header.magic, "PAK?", 4);    // 途中で落ちても壊れた pak を読ませない。完了時に "PACK" へ書き換え ❺
    ⋮
610     if (ext == ".mp3" || ext == ".MP3") {                                                        ❸
611         bool is3d = Is3DAudioPath(relPath);   // ← .pcm に変える前の .mp3 パスで判定
612         if (!DecodeMP3ToPCM(srcPath, fileData, is3d)) {
    ⋮
623     } else if (ext == ".png" || ext == ".PNG" ||
624         ext == ".hdr" || ext == ".HDR" ||
625         ext == ".jpg" || ext == ".JPG") {
626         if (!EncodeImageToDDS(srcPath, fileData)) {                                              ❹
```

asset_baker.cpp L63–L65 / L72–L78 / L581 / L610–L612 / L623–L626

速くなったかを数字で確かめる — 起動時間の計測

課題

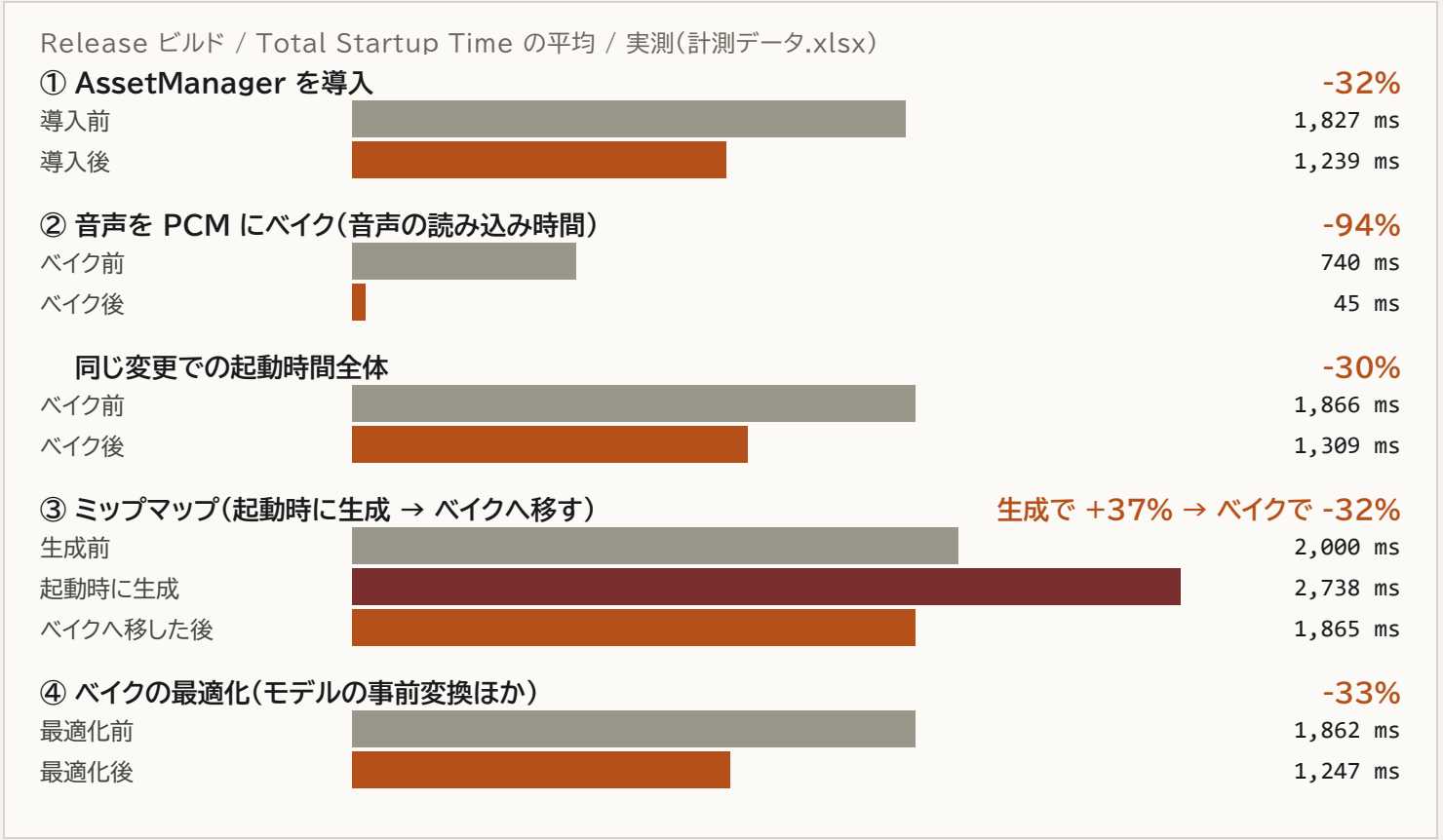
「速くなった気がする」で終わらせず、変更のたびに**起動時間を計測**して、効いたかどうかを数字で確かめたかった。

やったこと

各処理に `ScopedTimer` を入れ、起動の内訳（アセット読み込み・音声・シーン初期化）を毎回ログに出す。変更の前後で、**Release ビルドを同じ条件で3回前後**測って平均した。

わかったこと

- 音声のデコードが、起動の中でも重かった（約740ms）。
- ミップマップを**起動時に作ると、逆に遅くなった。**生成をベイクへ移すと、元の水準まで戻った。



衝突のルールをゲームから切り離す — レイヤーと関数表

課題

衝突判定のルールを、敵・弾・剣・地形などの**ゲームのロジックから切り離したい**。「この敵はあの弾と当たらない」を、各クラスの中の if で書くと散らかる。

やったこと

Unity の Physics Layer Matrix と同じ発想にした。GameObject は**自分のレイヤーを1つ**持つだけ。衝突するかどうかは、`PhysicsWorld` が持つ**レイヤー×レイヤーの2次元 bool 配列**で決める❷❸。

初期値は「全部当たる」にして、**例外だけを書く**。設定は対称になるよう、片方を書けば両方に効く。

形の組み合わせ(球・箱・OBB・カプセル)は、**4×4の関数表**で振り分ける❶。

ポイント

- 静的なレイヤー(地形・草木)は動かさず、動くほうだけを押し戻す。無駄な処理をしない。
- OBB×OBB は使う場面がないため未実装（`DispNotImpl`）。必要になった時に足す方針。

physics_world.cpp

```
559     IntersectFn s_Table[kTypeCount][kTypeCount] = {                                  ❶
560         //              b=Sphere,    b=Box,       b=OBB,        b=Capsule
561         /* a=Sphere  */ { DispSS,    DispSB,      DispSO,       DispSC },
562         /* a=Box     */ { DispBS,    DispBB,      DispBO,       DispBC },
563         /* a=OBB     */ { DispOS,    DispOB,      DispNotImpl,  DispOC },
564         /* a=Capsule */ { DispCS,    DispCB,      DispCO,       DispCC },
565     };
    ⋮
604     // PlayerAttack は Enemy とだけ衝突すればよい                                    ❷
605     SetLayerCollision(CollisionLayer::PlayerAttack, CollisionLayer::Player, false);        // Player本体には当たらない
606     SetLayerCollision(CollisionLayer::PlayerAttack, CollisionLayer::PlayerAttack, false);  // 複数剣同士も無効
607     SetLayerCollision(CollisionLayer::PlayerAttack, CollisionLayer::Terrain, false);       // 地形は貫通
608     SetLayerCollision(CollisionLayer::PlayerAttack, CollisionLayer::Foliage, false);       // 草木も貫通
    ⋮
616     // Foliage（草木）：Enemyとは絡まない（貫通させる）
617     SetLayerCollision(CollisionLayer::Foliage, CollisionLayer::Enemy, false);
618     SetLayerCollision(CollisionLayer::Foliage, CollisionLayer::EnemyBullet, false);
    ⋮
632 bool PhysicsWorld::CanCollide(Collider* a, Collider* b) const {                         ❸
633     return m_LayerMatrix[static_cast<int>(a->GetLayer())][static_cast<int>(b->GetLayer())];
634 }
```

physics_world.cpp L559–L565 / L604–L608 / L616–L618 / L632–L634

斜めの剣を判定する — OBB を「ローカル空間の AABB」に帰着させる

課題

剣の当たり判定は斜めに構える。AABB（軸に揃った箱）では、振りの向きに合わない余白が出て、当たっていないのに当たる。

やったこと

向きを持てる **OBB** を入れた。相手の球やカプセルを、**OBB のローカル座標へ移して**❶しまえば、OBB は「ただの AABB」になる。あとは、最近接点を箱の範囲へクランプするだけ❷。

カプセルは、まず芯の線分上で箱の中心に近い点を取り❸、箱へクランプし、もう一度線分へ射影する（2回の近似）。

結果は、ローカルの法線をワールドへ戻して返す。

ポイント

- OBB×球 は「ローカルへ移して AABB×球 に帰着」させるだけなので、新しい判定式を作らずに済んだ。
- 空間分割は入れていない。総当たりで足りている間は増やさず、**必要になったら実測してから**入れる方針（YAGNI）。

physics_world.cpp

```
185  float localX = d.x * ax.x + d.y * ax.y + d.z * ax.z;                    ❶
186  float localY = d.x * ay.x + d.y * ay.y + d.z * ay.z;
187  float localZ = d.x * az.x + d.y * az.y + d.z * az.z;
188
189  // OBBのハーフサイズにクランプ（最近接点をローカル空間で計算）
190  Vector3 hs = obb->GetHalfSize();
191  float cx = std::max(-hs.x, std::min(localX, hs.x));                      ❷
192  float cy = std::max(-hs.y, std::min(localY, hs.y));
193  float cz = std::max(-hs.z, std::min(localZ, hs.z));
     ⋮
352  Vector3 a1 = toLocal(cap->GetPointA());
353  Vector3 a2 = toLocal(cap->GetPointB());
354
355  // capsule core closest to the box center, then clamp to the box, then
356  // re-project onto the core (same 2-pass refinement as CapsuleVsBox)
357  Vector3 segPt = ClosestPointOnSegment(Vector3{ 0.0f, 0.0f, 0.0f }, a1, a2);  ❸
358  float cx = std::max(-hs.x, std::min(segPt.x, hs.x));
359  float cy = std::max(-hs.y, std::min(segPt.y, hs.y));
360  float cz = std::max(-hs.z, std::min(segPt.z, hs.z));
361  Vector3 boxPt = { cx, cy, cz };
362  segPt = ClosestPointOnSegment(boxPt, a1, a2);
```

physics_world.cpp L185–L193 / L352–L362

ボスが天高く飛んだ — 自分自身との衝突

起きたこと

ボスの当たり判定を実寸に合わせて、**1つの GameObject に 2つのコライダー**（下半身の Box と頭の Sphere）を持たせた。すると、ボスが天高く飛んでいった。

原因

物理の判定は、GameObject のペアではなく**コライダーのペア**で総当たりしていた。ボスの Box と Sphere は、設計上めり込んだ配置になるので、**同じボスのコライダー同士**が衝突と判定された。

押し戻しは「両方動く場合は半分ずつ」。同じオブジェクトに2回かかって互いに上書きし合い、位置が毎フレーム上へずれ続けた。

直したこと

ペアの先頭で、**同じ GameObject 同士は飛ばす**❶。1オブジェクト1コライダーを前提にしていた見落としだった。

なぜ2つに分けたか

下から突進されるとボスが浮いた。単一のカプセルは下端が丸く、低い位置への接触で押し戻しが斜め上に向いたため。下半身を平らな Box にして解消した。

enemy.cpp

```
105  auto* body = AddComponent<BoxCollider>();
106  body->SetSize({ bodyHalfWidth * 2.0f, bodyHeight, bodyHalfWidth * 2.0f });
107  body->SetOffset({ 0.0f, bodyHeight * 0.5f, 0.0f });
108  body->SetTag(CollisionTag::Enemy);
109  body->SetLayer(CollisionLayer::Enemy);
110
111  auto* head = AddComponent<SphereCollider>();
112  head->SetRadius(headRadius);
113  head->SetOffset({ 0.0f, bodyHeight, 0.0f });   // 肩(Boxの天井)に乗せる
114  head->SetTag(CollisionTag::Enemy);
```

enemy.cpp L105–L114

physics_world.cpp

```
659      GameObject* objB = colB->GetGameObject();
660      if (!objB || objB->IsDestroyed()) continue;
661
662      // 複数のColliderを持つGameObject同士で、同じGameObject同士の衝突は無視する
663      if (objA == objB) continue;                                          1
664
665      // Layerフィルタ
666      if (!CanCollide(colA, colB)) continue;
```

physics_world.cpp L659–L666

動きの繋がりを1つの表にする ― キャンセル表と先行入力

課題

アクションの手触りは、「どの動きから、どの動きへ、いつから移れるか」で決まる。条件を if で散らすと、調整のたびにコード中を探し回ることになる。

やったこと

すべてを**1つの表**に集約した❶。値は「その行動へ移れるようになるまでの秒数」。調整はこの表だけを見ればよい。

今の状態	ジャンプ	回避	攻撃	強攻撃
回避中	0.10	0.25	0.10	0.10
攻撃中(空振り)	0.28	0.28	0.28	0.28
被弾中	–	0.20	–	–
飛びつき中	–	0.15	–	–

その行動へ移れるまでの秒数(player.cpp L470–L502 の値)。― は移れない。

状態の変更は `SetState` だけを通し❷、経過時間のリセット漏れを防ぐ。攻撃は**空振りだと0.28秒**後まで派生できず、**命中したときだけ**0.04秒後から攻撃・強攻撃・回避へ繋げられる❹。早く押された入力は**0.26秒**保持し、受付が開いた瞬間に出す❸。

player.cpp

```
456      if (s != PlayerState::Attack) {
457          m_LungeTimer = 0.0f;
458          m_ComboAssistTimer = 0.0f;
459          m_AttackHitConfirmTimer = 0.0f;
460      }
461      m_State = s;                                                    ❷
462      m_StateTimer = 0.0f;
         ⋮
489      case PlayerState::Attack:
490          // 空振り時は簡単にキャンセルできない。命中時だけ下の特例で早く派生する。
491          return { 0.28f, 0.28f, 0.28f, 0.28f };                     ❶
         ⋮
505      if (m_State == PlayerState::Attack                              ❹
506          && m_AttackHitConfirmTimer > 0.0f
507          && m_StateTimer >= HIT_CONFIRM_CANCEL_DELAY
508          && (next == ActionKind::Attack || next == ActionKind::Heavy || next == ActionKind::Dodge)) {
509          return true;
         ⋮
527  void Player::BufferAction(ActionKind a) {
528      m_BufferedAction = a;
529      m_BufferTimer = INPUT_BUFFER_TIME;                              ❸
530  }
         ⋮
541      if (m_BufferTimer <= 0.0f) {          // 古くなった入力は捨てる
542          m_BufferedAction = ActionKind::None;
         ⋮
546      if (!CanCancelInto(m_BufferedAction)) return;   // まだ受付時間外。次フレームで再挑戦
```

player.cpp L456–L462 / L489–L491 / L505–L509 / L527–L530 / L541–L542 / L546

回避にリスクと見返りを — ジャスト回避

課題

回避の間ずっと無敵だと、連打すれば**永久に無敵**になる。
回避にリスクも見返りもなくなってしまう。

やったこと

無敵は、回避の**出だしの0.35秒**だけにした❶（回避全体は0.5秒）。その間に当たったら「ジャスト回避」とする❷。

成功すると、**反撃の猶予が1.2秒**できて与ダメージが3倍（通常は1）になる❸。同時に、時間が0.45秒だけ0.25倍になり、画面の白フラッシュと専用の演出を出す。

ポイント

- 判定を先読みしない。`TakeDamage` に届いた時点で**「本当に当たっていた」**ことが保証されているので、その瞬間に無敵時間中かを見るだけでよい。
- スローは、ヒットストップと同じ仕組み（時間の進み方を変える）を使い回している。入力は実時間なので、自分だけ速く動ける感覚になる。

player.h

```
173  static constexpr float DODGE_IFRAME = 0.35f;   // 回避の無敵時間（回避開始から）   ❶
174  static constexpr float COUNTER_TIME = 1.20f;   // ジャスト回避後の反撃猶予
175  static constexpr int   COUNTER_DAMAGE = 3;     // 反撃の与ダメージ（通常は1）
176  static constexpr float SLOWMO_TIME = 0.45f;    // スロー演出の長さ
177  static constexpr float SLOWMO_SCALE = 0.25f;   // スロー中の時間の進み方
     ⋮
418  bool IsDodgeInvincible() const {
419      return m_State == PlayerState::Dodge && m_StateTimer <= DODGE_IFRAME;
420  }
```

player.h L173–L177 / L418–L420

player.cpp

```
1082  if (IsDodgeInvincible()) {                                                    ❷
1083      OnPerfectDodge();
1084      return;
1085  }
      ⋮
1264  m_CounterTimer = COUNTER_TIME;                                                ❸
1265
1266  // 世界だけ遅くする。プレイヤーの入力は実時間なので「自分だけ速い」感覚になる
1267  HitStop::Trigger(SLOWMO_TIME, SLOWMO_SCALE);
```

player.cpp L1082–L1085 / L1264–L1267

避けられる攻撃にする ― 予兆と当たり判定

課題

敵の攻撃に予告が無いと、ジャスト回避が**運になる**。かといって、画面が光りすぎても遊びの邪魔になる。

やったこと

攻撃の溜めの間だけ、**頭上のグロー**と**体の色**で予告する。時間に応じて大きくし❶、終盤は点滅を速める❷。色は攻撃の種類ごとに変えた（近接＝橙、射撃＝赤、ボスの叩きつけ＝黄）。

体の色は、元の色と警告色を**混ぜる**（ピークで約5割）❸。発射の直前（残り20%）には、白いフラッシュで「今」を知らせる❹。

攻撃の当たり判定

近接の攻撃は、前方に**箱の当たり判定を生成**する❺。以前は水平の距離だけで判定していて、ビルの上のプレイヤーにも当たっていた。箱にしたので、高さも見る。

enemy.cpp

```
201  float t = m_StateTimer / startup;              // 0→1
202  float grow = 0.25f + 0.75f * (t * t);          // 出だしから見えるように下駄をはかせる   ❶
     ⋮
205  float blink = 1.0f;
206  if (t > 0.45f) {
207      float f = (t - 0.45f) / 0.55f;
208      blink = 0.5f + 0.5f * sinf(m_StateTimer * (26.0f + 55.0f * f));   ❷
209  }
     ⋮
240      // 元のボディ色と警告色を半々くらいで混ぜる（ピークで約50%）
241      float mix = (0.10f + 0.40f * grow) * (0.65f + 0.35f * blink);   ❸
242      model->SetTint(true, { cr, cg, cb, 1.0f }, mix);
     ⋮
246  if (m_TelegraphCore) {
247      if (t > 0.80f) {                                                ❹
248          float s = (t - 0.80f) / 0.20f;                  // 0→1
249          float cs = TELEGRAPH_CORE_MAX * scale * (0.35f + 0.65f * s);
250          m_TelegraphCore->SetSize(cs, cs);
251          float w = 2.0f + 5.0f * s;    // スナップも控えめ
```

enemy.cpp L201–L202 / L205–L209 / L240–L242 / L246–L251

enemy_attack.cpp

```
13  auto* col = AddComponent<BoxCollider>();                          ❺
14  col->SetSize(m_BoxSize);
15  col->SetOffset({ 0.0f, 0.0f, 0.0f });
16  col->SetLayer(CollisionLayer::EnemyBullet);   // 敵の攻撃用レイヤー(Enemy とは当たらない設定済み)
17  col->SetTrigger(true);                          // 押し戻さない。当たり判定だけ
     ⋮
38  if (Player* player = dynamic_cast<Player*>(info.other)) {
39      m_Hit = true;
40      player->TakeDamage(m_Damage, m_KnockDir);
41  }
```

enemy_attack.cpp L13–L17 / L38–L41

モデルを使わず、曲線から武器を組み立てる — 虚装

課題

「体から伸びる影の武器（虚装）」は形が決まっていない。固定のメッシュだと、動きの表現に合わない。

やったこと

モデルを持たず、**毎フレーム頂点を組み立てて**描く。振り・溜め・突きは、**2次ベジエ曲線の制御点を動かすだけ**で表す❶。物理シミュレーションは持たない。

曲線の各点に**断面の座標系**を運び❷、平たい楕円の断面を掃引して**チューブ**にする。カメラ正対の板にすると、どこから見ても厚みが無くなるため。

ポイント

- 長さ方向に等間隔の**瘤**を立てて節を作る❸。
- 周期の違う正弦を掛け合わせた**痙攣**を重ねる❹。単純な正弦1本だと、きれいに振動しているだけに見えるため。
- シェーダーはライティング無し（UNLIT）なので、**陰影を頂点カラーに焼き込む**（ランバート・リム・縁）。

phantom_arm.cpp

```
31  static Vector3 Bezier(const Vector3& p0, const Vector3& p1, const Vector3& p2, float t) {
32      const float u = 1.0f - t;
33      return p0 * (u * u) + p1 * (2.0f * u * t) + p2 * (t * t);                    ❶
34  }
    ⋮
330             // 前の U から接線成分を抜いて、直交させ直す
331             U = carriedU - tan * Vector3::dot(carriedU, tan);                    ❷
    ⋮
349         float barb = sinf(u * BARB_FREQ + phase * 1.7f);
350         barb = (barb > 0.0f) ? powf(barb, BARB_SHARP) : 0.0f;
351         const float swell = 1.0f + BARB_AMP * barb;                              ❸
352
353         const float hw = pt.halfWidth * swell;              // 幅（刃の広い方）
354         const float ht = pt.halfWidth * TUBE_THICK * swell; // 厚み（刃の薄い方）
    ⋮
362         const float twEnv = powf(u, 0.70f) * m_Deploy;
363         const float twitch = sinf(u * TWITCH_FREQ + m_Time * TWITCH_SPEED + phase * 3.1f)   ❹
364             * sinf(m_Time * TWITCH_BEAT + phase)
365             * TWITCH_AMP * twEnv;
366
367         const Vector3 center = pt.pos + U * twitch + V * (twitch * 0.6f);
```

phantom_arm.cpp L31–L34 / L330–L331 / L349–L354 / L362–L367

手応えを組み合わせで作る — ヒットストップと被弾演出

課題

当たった手応えは、ダメージ数字ではなく、画面と音と振動で決まる。1つの演出だけ強くしても、まとまった手応えにならない。

やったこと

ヒットストップは、時間の進み方（`TimeScale`）を一瞬だけ下げて作る❶。カウントダウンは`UnscaledDeltaTime`で行うので、止まっている間も戻る時刻がずれない。重なったときは、長いほうを採る❷。

被弾のときは、ヒットストップ・カメラの揺れ・パッドの振動・カメラごと押される動き・画面の効果・音を、**同じ入口から同時に**出す❸。斬撃・ジャスト回避・覚醒でも、同じ部品を強さだけ変えて使う。

ポイント

- 演出は「フレームワーク層」の部品（`engine/feedback`）。ゲーム側は強さと長さを渡すだけ。

hit_stop.cpp

```
17  void HitStop::Trigger(float duration, float scale) {
18      if (s_Timer <= 0.0f) {
19          // 新規発動：現在のTimeScaleを保存
20          s_OriginalScale = Time::GetTimeScale();
21      }
22      // 既存のヒットストップより長ければ上書き（重なった時の対応）
23      if (duration > s_Timer) {                                   ❷
24          s_Timer = duration;
25      }
26      Time::SetTimeScale(scale);                                  ❶
27  }
    :
33      s_Timer -= Time::GetUnscaledDeltaTime();
```

hit_stop.cpp L17–L27 / L33

player.cpp

```
1110  HitStop::Trigger(0.15f, 0.05f);                               ❸
1111  CameraShake::Trigger(1.2f, 0.42f);
1112  InputManager::StartVibration(0.8f, 0.32f);    // 画面の揺れと同じ強さ・長さで振動させる
1113  CameraPush::Trigger(knockbackDir, 0.7f, 0.30f);    // 殴られた方向へカメラごと押される
1114  ScreenFX::TriggerDamage(1.0f, 0.95f);
1115  ScreenFX::TriggerFlash(0.18f, 0.09f);    // 一瞬白く抜いて食らった瞬間を強く
1116  Audio_PlaySE3D(AudioID::SE_DAMAGE, m_Position.x, m_Position.y, m_Position.z);
```

player.cpp L1110–L1116

担当のまとめと、次の課題

159
ソースファイル数

26,966
ソース行数(.cpp / .h)

29
HLSL シェーダー

約1.25秒
Release の起動時間(以前は約1.86秒)

担当の要点

- 個人開発として、フレームワーク(描画・物理・アセット・音)からゲームの中身まで設計・調整した。コードの記述は主に AI(Claude Code)に依頼し、仕様の指示と動作確認は私が行った。
- 「追加・調整が1か所で済む形」を優先した(X-Macro、レイヤーの表、キャンセル表)。
- 速度の話は**計測してから**手を入れ、効かなかったものも記録に残した。

次に手を入れたいところ

- **衝突判定の総当たり:** 現状は全コライダーの組み合わせを調べている(physics_world.cpp L647〜)。敵の数を増やすなら、空間分割を実測とセットで入れる。
- **OBB×OBB:** 未実装のまま。武器同士の判定など、必要になった時に足す。
- **ファイルの大きさ:** `player.cpp` は約1,500行、`phantom_arm.cpp` は約1,300行ある。状態の管理と演出を分けたい。